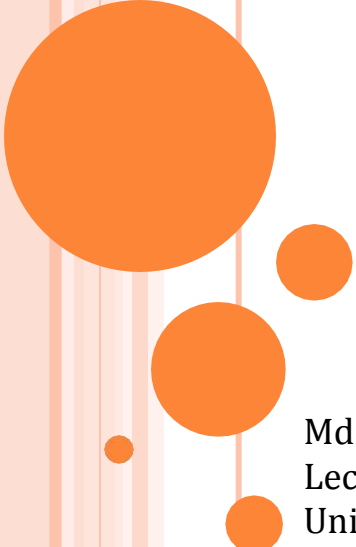


OOP PRINCIPLES

CSE 1115: OBJECT ORIENTED PROGRAMMING



Md. Shafqat Talukder Rakin
Lecturer, Department of CSE,
United International University
Email id : shafqat@cse.uiu.ac.bd

Acknowledged: Tanjina Helaly

3 PRINCIPLES

- All object-oriented programming languages provide mechanisms that help you implement the object-oriented model.
 - Inheritance
 - Encapsulation
 - Polymorphism



INHERITANCE



INHERITANCE

- ❑ Inheritance is the process by which one object acquires the properties of another object.
- ❑ It is a way to form new classes using classes that have already been defined.
- ❑ The new classes (child classes), take over (or inherit) **attributes** and **behavior** of the pre-existing classes (parent classes).



INHERITANCE

- Java uses “extends” keyword to show inheritance relationship. Example
class Child extends Parent{}
- The class that is extended is a **superclass**
 - Other terms: parent class, base class, ancestor class
 - The extended class is a **subclass** of its superclass
 - Other terms: child class, extended class, derived class
- An object created from the subclass has its **own copy** of all the **nonstatic fields** defined in its superclass



INHERITANCE

- ❑ This is important because it supports the concept of hierarchical classification.
- ❑ Inheritance provides a powerful and natural mechanism for organizing and structuring your software
- ❑ It is intended to help reuse existing code with little or no modification.



INHERITANCE - EXAMPLE

```
class Parent {
    public int parentVariable = 10;
    public void parentMethod() {
        System.out.println( "Parent Method" );
    }
}

class Child extends Parent {
    public int childVariable = 5;
    public void childMethod() {
        parentMethod();
        System.out.printf( "In Child ParentVariable=%d, ChildVariable=%d",
            parentVariable, childVariable );
    }
}

class Inheritance {
    public static void main( String args[] ) {
        Child example = new Child();
        example.childMethod();
        example.parentMethod();
        System.out.println( example.parentVariable );
    }
}
```



INHERITANCE - EXAMPLE

```
class Parent {
    public int parentVariable = 10;
    public void parentMethod() {
        System.out.println( "Parent Method" );
    }
}

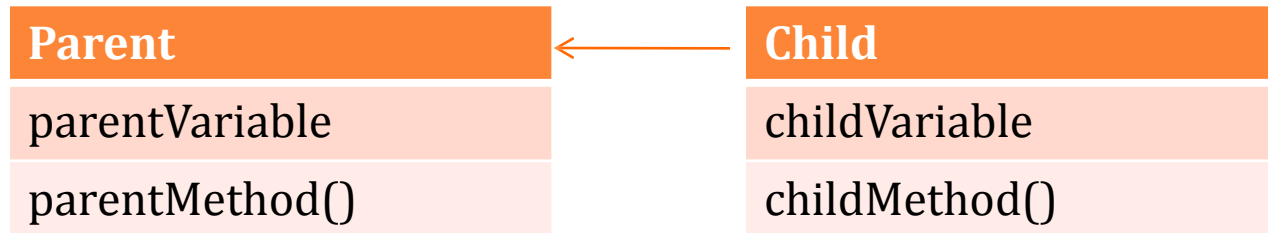
class Child extends Parent {
    public int childVariable = 5;
    public void childMethod() {
        parentMethod();
        System.out.printf( "In Child ParentVariable=%d, ChildVariable=%d",
            parentVariable, childVariable );
    }
}

class Inheritance {
    public static void main( String args[] ) {
        Child example = new Child();
        example.childMethod();
        example.parentMethod();
        System.out.println( example.parentVariable );
    }
}
```

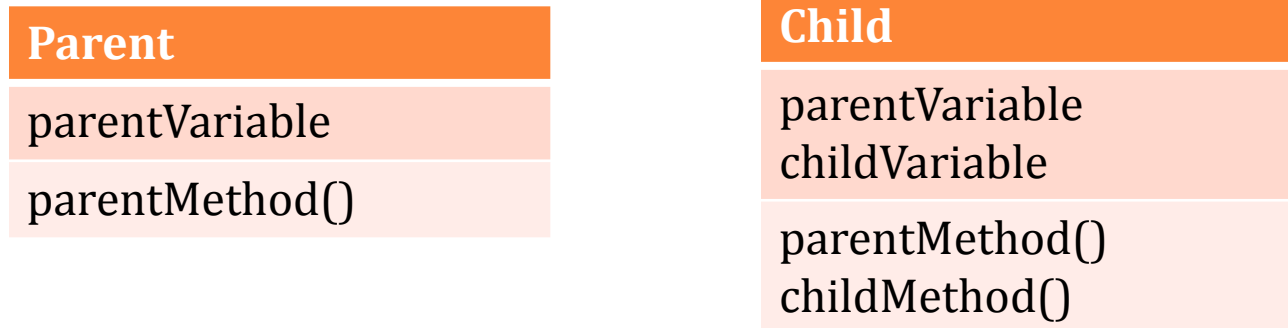
Output

```
Parent Method
In Child ParentVariable=10,
ChildVariable=5 Parent Method
10
```


INHERITANCE - EXAMPLE



Act like



TYPES OF INHERITANCE

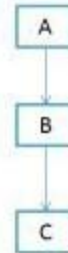
□ Single inheritance

- One parent-> One child



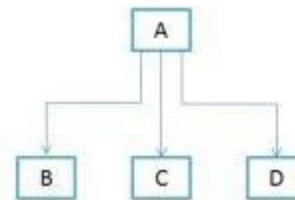
□ Multi level inheritance

- one can inherit from a derived class, thereby making this derived class the base class for the new class.



□ Hierarchical inheritance

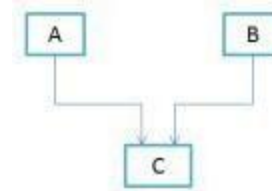
- One parent – multiple children



TYPES OF INHERITANCE

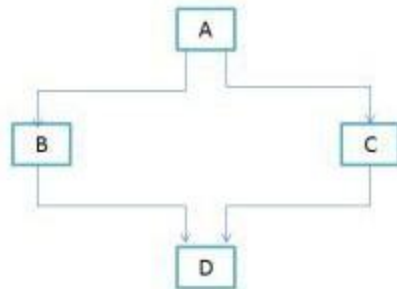
- Multiple inheritance

- Multiple parents -> one child



- Hybrid inheritance

- Is a combination of Single and Multiple inheritance



- Java does not support multiple inheritance



INHERITANCE – SUPER KEYWORD

- The **super** keyword in java is a reference variable that is used to refer immediate parent class object.
- Whenever you **create the instance of subclass, an instance of parent class is created implicitly** i.e. referred by super reference variable.
- Usage of java super Keyword
 - super is used to refer immediate parent class member(instance variable & method).
 - super() is used to invoke immediate parent class constructor.



SUPER – PARENT'S INSTANCE VARIABLE/METHOD

- The super keyword can also be used to
 - Access parent class's instance variable if child has an instance variable with same name.
 - invoke parent class's method. It should be used in case subclass contains the same method as parent class.



SUPER – PARENT'S INSTANCE VARIABLE/METHOD

```
class Parent {  
    String name;  
    void message(){  
        System.out.println("Welcome to Parent class."); }  
}  
// Create a subclass by extending class parent.  
class Child extends Parent {  
    String name; // this name hides the name in Parent  
  
    public Child(String a, String b) {  
        super.name = a; // name in A  
        name = b; // name in B  
    }  
    void show() {  
        System.out.println("superclass's name: " + super.name + "; subclass's name: " + name); }  
  
    //method overriding  
    void message(){  
        System.out.println("Welcome to Child class."); }  
  
    void display(){  
        message();//will invoke current class message() method  
        super.message();//will invoke parent class message() method  
    }  
}
```



SUPER – PARENT'S INSTANCE VARIABLE/METHOD

```
public class TestSuper {  
    public static void main(String[] args) { Child  
        s=new Child("Parent", "Child");  
        s.show();  
        s.display();  
    }  
}
```

□ Output:

superclass's name: Parent; subclass's name: Child

Welcome to Child class.

Welcome to Parent class.



SUPER-TO INVOKE PARENT CONSTRUCTOR.

- The `super` keyword can also be used to invoke the parent class constructor as given below:

```
class Vehicle{
    Vehicle(){
        System.out.println("Vehicle is created.");
    }
}

class Bike extends Vehicle{
    Bike(){
        super(); //will invoke Parent class's constructor
        System.out.println("Bike is created.");
    }
    public static void main(String[] args){
        Bike b = new Bike();
    }
}
```

Output:
Vehicle is created
Bike is created



SUPER-TO INVOKE PARENT CONSTRUCTOR.

- What would be the output of the program below:

```
class Vehicle{  
    Vehicle(){  
        System.out.println("Vehicle is created.");  
    }  
}
```

```
class Bike extends Vehicle{  
    Bike(){  
        System.out.println("Bike is created.");  
    }  
    public static void main(String[] args){  
        Bike b = new Bike();  
    }  
}
```



SUPER-TO INVOKE PARENT CONSTRUCTOR.

- What would be the output of the program below:

```
class Vehicle{
    Vehicle(){
        System.out.println("Vehicle is created.");
    }
}

class Bike extends Vehicle{
    Bike(){
        System.out.println("Bike is created.");
    }
    public static void main(String[] args){
        Bike b = new Bike();
    }
}
```

Output:
Vehicle is created
Bike is created

If the constructor doesn't have super(), compiler will provide super() as the first statement of the constructor.



SUPER-TO INVOKE PARENT CONSTRUCTOR.

□ **Note:**

- `super()` should be the first statement of your constructor
- If the constructor doesn't have `super()`, compiler will provide `super()` as the first statement of the constructor.
- **It will only work when Parent has no constructor or a parameter less constructor. See the next example.**



SUPER-TO INVOKE PARENT CONSTRUCTOR.

- What would be the output of the program below:

```
class Vehicle{  
    Vehicle(String name){  
        System.out.printf("Vehicle %s is created.\n", name);  
    }  
}  
  
class Bike extends Vehicle{  
    Bike(){  
        System.out.println("Bike is created.");  
    }  
  
    public static void main(String[] args){  
        Bike b = new Bike();  
    }  
}
```



SUPER-TO INVOKE PARENT CONSTRUCTOR.

- What would be the output of the program below:

```
class Vehicle{
    Vehicle(String name){
        System.out.printf("Vehicle %s is created.\n", name);
    }
}

class Bike extends Vehicle{
    Bike() // Implicit super constructor Vehicle() is undefined. Must explicitly
    invoke another constructor
        System.out.println("Bike is created.");
}

public static void main(String[] args){
    Bike b = new Bike();
}
}
```



SUPER-TO INVOKE PARENT CONSTRUCTOR.

□ So the fix is:

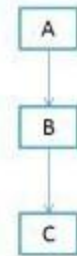
```
class Vehicle{  
    Vehicle(String name){  
        System.out.printf("Vehicle %s is created.\n", name);  
    }  
}
```

```
class Bike extends Vehicle{  
    Bike(){  
        super("Bike");  
        System.out.println("Bike is created.");  
    }  
  
    public static void main(String[] args){  
        Bike b = new Bike();  
    }  
}
```



WHEN CONSTRUCTORS ARE EXECUTED

- When a class hierarchy is created, the constructors are executed in the order of the hierarchy.



- For example: for the hierarchy above, constructor of A will be executed first, then B and then C.



WHEN CONSTRUCTORS ARE EXECUTED

```
// Create a super class.
class A {
    A() {
        System.out.println("Inside A's constructor."); }
}
// Create a subclass by extending class A.
class B extends A {
    B() {
        System.out.println("Inside B's constructor."); }
}
// Create another subclass by extending B.
class C extends B {
    C() {
        System.out.println("Inside C's constructor."); }
}
class CallingCons {
    public static void main(String args[]) { C
        c = new C();
    }
}
```

Output:

Inside A's constructor
Inside B's constructor
Inside C's constructor



OBJECT CLASS

- All classes inherit directly or indirectly from `java.lang.Object`
 - `class Parent { int size; }`
 - `class Parent extends Object { int size; }`
- The child class below is a grandchild of `Object`
 - `class Child extends Parent{ }`
- Having a common ancestor class allows java to provide standard members on all objects, like `toString()`

```
class TestObject {  
    public static void main( String args[] ) {  
        Parent watchThis = new Parent(); int  
        myHash = watchThis.hashCode();  
        System.out.println( myHash );  
        // Where does hashCode come from?  
    }  
}
```



OBJECT CLASS - METHODS

- `clone()` - Creates a clone of the object.
- `equals(Object)` - Compares two Objects for equality.
 - Uses "==" to test for equality
 - Note: If a class needs an implementation of equals, which differs from the default "equals", the class should override the method.
- `toString()` - Returns a String that represents the value of this Object.



Spring 2024 Question 1

QUESTION 1

[3+2+2.5 MARKS]

Consider the following codes:

```
class Point2D
{
    int x, y;
    public Point2D(int x, int y){
        this.x = x;
        this.y = y;
        System.out.println("Point2D constructor");
    }
    public String Display(){
        //write codes here
    }
}
class Point3D extends Point2D{
    int z;
    //write codes here
}
```

```
public class Test{
    public static void main(String args[]){
        Point2D p2D = new Point2D(1, 2);
        System.out.println(p2D.Display());
        Point3D p3D = new Point3D(5, 4, 3);
        System.out.println(p3D.Display());
    }
}
```

Now:

- I. Complete the “Display” method of the Point2D class that prints all the instance variables,
 - II. Add a constructor in the Point3D class that uses the base class constructor,
 - III. Add another method “Display” in the Point3D class. You have to use the parent’s “Display” method here,
- so that the **output** looks like this:

```
Point2D constructor
x= 1, y= 2
Point2D constructor
Point3D constructor
x = 5, y = 4, z = 3
```



Spring 2024 Question 1

i)

```
public String Display() { 3 usages 1 override
    String s = new String( original: "x = " + x + ", y = " + y);
    return s;
}
```

ii)

```
public Point3D(int x, int y, int z) { 1 usage
    super(x, y);
    this.z = z;
    System.out.println("Point3D constructor");
}
```

iii)

```
public String Display() { 3 usages
    return super.Display() + ", z = " + z;
}
```



Summer 2023 Question 1

Q1. Write the output of the following program.

[5]

<pre>public class Vehicle { private String brand; private String model; static { System.out.println("Initializing Vehicle class..."); } { System.out.println("Initializing an instance of the Vehicle class..."); } public Vehicle() { System.out.println("Creating a default vehicle."); brand = "Unknown"; } public Vehicle(String brand, String model) { System.out.println("Creating a customized vehicle of brand: " + brand + " and model: " + model); this.brand = brand; this.model = model; } public void honk() { System.out.println("The vehicle emits a honking sound."); } }</pre>	<pre>public void honk(String sound) { System.out.println("The vehicle emits a custom honking sound: " + sound); } static { System.out.println("Making sure of initialization..."); } public void info() { System.out.println("model="+model+" brand="+brand); } public static void main(String[] args) { Vehicle defaultVehicle = new Vehicle(); defaultVehicle.honk(); defaultVehicle.info(); Vehicle truck = new Vehicle("Ford", "F- 150"); truck.honk("Loud horn sound"); truck.info(); } }</pre>
--	--



Summer 2023 Question 1

Q1. Write the output of the following program.

[5]

<pre>public class Vehicle { private String brand; private String model; static { System.out.println("Initializing Vehicle class..."); } { System.out.println("Initializing an instance of the Vehicle class..."); } public Vehicle() { System.out.println("Creating a default vehicle."); brand = "Unknown"; } public Vehicle(String brand, String model) { System.out.println("Creating a customized vehicle of brand: " + brand + " and model: " + model); this.brand = brand; this.model = model; } public void honk() { System.out.println("The vehicle emits a honking sound."); } }</pre>	<pre>public void honk(String sound) { System.out.println("The vehicle emits a custom honking sound: " + sound); } static { System.out.println("Making sure of initialization..."); } public void info() { System.out.println("model="+model+" brand="+brand); } public static void main(String[] args) { Vehicle defaultVehicle = new Vehicle(); defaultVehicle.honk(); defaultVehicle.info(); Vehicle truck = new Vehicle("Ford", "F- 150"); truck.honk("Loud horn sound"); truck.info(); }</pre>
--	--

Initializing Vehicle class...

Making sure of initialization...

Initializing an instance of the Vehicle class...

Creating a default vehicle.

The vehicle emits a honking sound.

model=null brand=Unknown

Initializing an instance of the Vehicle class...

Creating a customized vehicle of brand: Ford and model: F- 150

The vehicle emits a custom honking sound: Loud horn sound

model=F- 150 brand=Ford



Summer 2023 Question 2

Q2. Consider the following class named `ElectronicDevice` representing a generic electronic device.
[5]

```
public class ElectronicDevice {  
    String brand;  
    double price;  
  
    public ElectronicDevice(String brand, double price) {  
        this.brand = brand;  
        this.price = price;  
    }  
  
    public void displayInfo() {  
        System.out.println("Brand: " + brand);  
        System.out.println("Price: $" + price);  
    }  
}
```



Summer 2023 Question 2

Now write the necessary codes to fulfill the requirements as follows:

1. Create a class named Smartphone - a child class of ElectronicDevice.
2. The Smartphone class has three additional attributes: model (String), operatingSystem (String), and IMEI (String). It must not be possible to set the value of IMEI outside of the class.
3. Create a constructor in Smartphone that takes brand, price, model, operatingSystem, and IMEI as arguments and sets the values of the attributes. This constructor invokes the constructor of ElectronicDevice.
4. There is a method named *displayInfo* in SmartPhone that shows the brand, price, model, and operatingSystem. The method invokes the *displayInfo* of ElectronicDevice.
5. There must be an option to fetch the IMEI outside of the class.



Summer 2023 Question 2

```
class ElectronicDevice { 1 usage 1 inheritor
    String brand; 2 usages
    double price; 2 usages

    public ElectronicDevice(String brand, double price) { 1 usage
        this.brand = brand;
        this.price = price;
    }

    public void displayInfo() { 2 usages 1 override
        System.out.println("Brand: " + brand);
        System.out.println("Price: $" + price);
    }
}

class Smartphone extends ElectronicDevice { 2 usages
    private String model; 2 usages
    private String operatingSystem; 2 usages
    private String IMEI; 2 usages

    public Smartphone(String brand, double price, String model, String operatingSystem, String IMEI) { 1 usage
        super(brand, price);
        this.model = model;
        this.operatingSystem = operatingSystem;
        this.IMEI = IMEI;
    }
}
```

Summer 2023 Question 2

```
@Override 2 usages
public void displayInfo() {
    super.displayInfo();
    System.out.println("Model: " + model);
    System.out.println("Operating System: " + operatingSystem);
}
```

```
public String getIMEI() { 1 usage
    return IMEI;
}
```

```
}
```

 Rename usages

```
public class Question_2 {
    public static void main(String[] args) {
        Smartphone phone = new Smartphone( brand: "Samsung", price: 999.99, model: "Galaxy S21", operatingSystem: "Android", IMEI: "123456789");
        phone.displayInfo();
        System.out.println("IMEI: " + phone.getIMEI());
    }
}
```



ENCAPSULATION



ENCAPSULATION

- ▣ *Encapsulation* is the mechanism that binds together code and the data it manipulates and keeps both safe from outside interference and misuse.
- ▣ One way to think about encapsulation is as a protective wrapper
 - that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.
- ▣ the basis of encapsulation is the class.
 - Use access modifier to provide encapsulation



ENCAPSULATION

- Since the **purpose** of a class/encapsulation is to encapsulate complexity, there are mechanisms for hiding the complexity of the implementation inside the class.
 - Use **access modifier** to specify which members should/ shouldn't be accessed by outside world.
 - the **public interface** should be carefully designed not to expose too much of the inner workings of a class



GETTER/SETTER METHOD

- In encapsulation, the variables of a class will be hidden from other classes, and can be accessed only through the methods of their current class.
- To achieve that in Java –
 - Declare the variables of a class as private.
 - Provide public setter and getter methods to modify and view the variables values.
 - Example:

```
private double balance;  
public void setBalance(double b) {  
    balance = b;  
}  
public double getBalance() {  
    return balance;  
}
```



BENEFIT OF ENCAPSULATION

□ Benefit

- Hide complexity from user.
- Implement logic while updating field
- Easy to maintain
- Make variable read-only or write only



POLYMORPHISM



POLYMORPHISM

- ❑ **Poly** means **many** and **morph** means **form**. Thus, polymorphism refers to being able to use many forms of a type without regard to the details.
- ❑ More generally, the concept of polymorphism is often expressed by the phrase “one interface, multiple methods.”
- ❑ *polymorphism* refers to a programming language's ability to process objects differently depending on their data type or class.



POLYMORPHISM

- 3 types
 - Method overriding
 - Method overloading
 - Subclass polymorphism



METHOD OVERRIDING

- Means a **child class** is **re-implementing** a **method of its super class**. Or
- A class replacing an ancestor's implementation of a method with an implementation of its own.
- When overriding a method in child class
 - Method Signature(name and argument list) and return type must be the same as the parent method.
 - Child method could be equal or more accessible.



METHOD OVERRIDING

- When an **overridden method is called** from within its **subclass**, it will always refer to the version of **that method defined by the subclass**.
 - The version of the method defined by the superclass will be hidden.
- To invoke parent method in child class, need to use “**super**” keyword.
- Static methods can not be overridden
- Dynamic binding



METHOD OVERRIDING - EXAMPLE

```
public class TestOverriding {  
    public static void main(String[] args) {  
        System.out.println("-----Parent-----");  
        Parent p = new Parent();  
        p.display();  
  
        System.out.println("\n-----Child-----");  
        Child c = new Child();  
        c.display();  
    }  
}  
  
class Parent {  
    public void display() {  
        System.out.println( "Display in Parent" );  
    }  
}  
  
class Child extends Parent {  
    public void display() {  
        System.out.println( "Display in Child" );  
    }  
}
```

Output:
-----Parent-----
Display in Parent

-----Child-----
Display in Child



METHOD OVERRIDING - EXAMPLE(SUPER KEYWORD)

```
public class TestOverriding {  
    public static void main(String[] args) {  
        System.out.println("-----Parent-----");  
        Parent p = new Parent();  
        p.display();  
  
        System.out.println("\n-----Child-----");  
        Child c = new Child();  
        c.display();  
    }  
}  
  
class Parent {  
    public void display() {  
        System.out.println("Display in Parent");  
    }  
}  
  
class Child extends Parent {  
    public void display() {  
        super.display(); // invoke the parent class's method  
        System.out.println("Display in Child");  
    }  
}
```

Output:
-----Parent-----
Display in Parent

-----Child-----
Display in Parent
Display in Child



METHOD OVERLOADING

- Two methods in the **same class** can have the **same name** with different **signature**. This is called method overloading and the methods are called overloaded method.
 - The signature of a method is its **name** with **number**, **type** and **order** of its parameters.
 - The return type is not part of the signature of a method.
- Condition:
 - Methods should be in the same class
 - Same name
 - Must have different argument.
 - Return type could be same or different.



METHOD OVERLOADING

- This called static binding because, which method to be invoked will be decided at the time of compilation



METHOD OVERLOADING - EXAMPLE

```
class OverloadDemo {  
    void test() {  
        System.out.println("No parameters"); }  
  
    // Overload test for one integer parameter.  
    void test(int a) {  
        System.out.println("a: " + a); }  
  
    // Overload test for a double parameter  
    double test(double a) {  
        System.out.println("double a: " + a);  
        return a*a; }  
}  
class Overload {  
    public static void main(String args[]) {  
        OverloadDemo ob = new OverloadDemo();  
        // call all versions of test()  
        ob.test();  
        ob.test(10);  
        double result = ob.test(123.25); System.out.println("Result  
of ob.test(123.25): " + result);  
    }  
}
```

Output:

No parameters

a: 10

double a: 123.25

Result of ob.test(123.25): 15190.5625



CONSTRUCTOR OVERLOADING

- ❑ In addition to overloading normal methods, you can also overload constructor methods.
- ❑ In fact, for most real-world classes that you create, overloaded constructors will be the norm, not the exception.



CONSTRUCTOR OVERLOADING - EXAMPLE

```
public class Box {  
    double width, height, depth;  
    // constructor used when all dimensions specified  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
    // constructor used when cube is created  
    Box(double len) {  
        width = height = depth = len;  
    }  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
    public static void main(String[] args){  
        Box b = new Box(10, 8, 5); System.out.println("Volume of  
        Box: " + b.volume()); Box b1 = new Box(5);  
        System.out.println("Volume of Cube: " + b1.volume());  
    }  
}
```

Output:
Volume of Box: 400.0
Volume of Cube: 125.0



INVOKING OVERLOADED CONSTRUCTORS -THIS()

- Sometimes it is useful for one constructor to invoke another.
 - this is accomplished by using the **this keyword**.
 - The general form is
 - *this(arg-list)*
- When **this()** is executed, the overloaded constructor that matches the parameter list specified by *arg-list* is executed.
- The call to **this()** must be the first statement **within** the constructor.
 - General rule: Constructor call must be the first statement in a constructor.



CONSTRUCTOR OVERLOADING - EXAMPLE

```
public class Box {  
    double width, height, depth;  
    // constructor used when all dimensions specified  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
    // constructor used when cube is created  
    Box(double len) {  
        width = height = depth = len;  
    }  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
    public static void main(String[] args){  
        Box b = new Box(10, 8, 5); System.out.println("Volume of  
        Box: " + b.volume()); Box b1 = new Box(5);  
        System.out.println("Volume of Cube: " + b1.volume());  
    }  
}
```

Output:
Volume of Box: 400.0
Volume of Cube: 125.0



CONSTRUCTOR OVERLOADING – EXAMPLE WITH THIS()

```
public class Box {  
    double width, height, depth;  
    // constructor used when all dimensions specified  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
    // constructor used when cube is created  
    Box(double len) {  
        this(len, len, len);  
    }  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
    public static void main(String[] args){  
        Box b = new Box(10, 8, 5); System.out.println("Volume of  
        Box: " + b.volume()); Box b1 = new Box(5);  
        System.out.println("Volume of Cube: " + b1.volume());  
    }  
}
```

Output:
Volume of Box: 400.0
Volume of Cube: 125.0



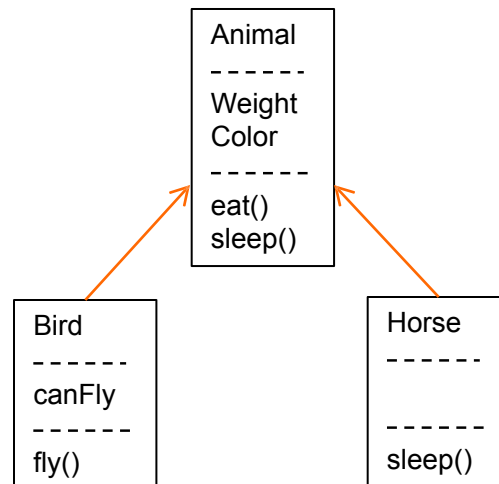
SUBCLASS POLYMORPHISM

- A **parent class reference** is used to **refer** to a **child class object**.
- Couple things to remember:
 - The only possible way to access an object is through a reference variable.
 - The type of the reference variable would determine the methods that it can invoke on the object.
- Using subclass polymorphism we can call or execute the child-class overriding method by the parent-class object.



SUBCLASS POLYMORPHISM - EXAMPLE

```
public class TestPolymorphism {  
    public static void main(String[] args) {  
        Animal a = new Bird();  
        a.sleep(); // sleep() method of animal class will be executed  
  
        Animal h = new Horse();  
        h.sleep(); // sleep() method of Horse class will be executed  
    }  
}
```



SUBCLASS POLYMORPHISM –COMPILE/RUN TIME

□ 2 types of check

- reference variable would determine the methods that it can invoke on the object. **Compile time check.**
- object type (NOT reference variable type) determines which overridden method will be used at **runtime.**
- Consider the code below
 - ```
Animal a = new Bird();
a.sleep();

Animal h = new Horse();
h.sleep();
```
  - Can't call a.sleep() if the sleep() method is not available in Animal class. Will produce compile error.
  - During runtime the Horse's sleep() method will be executed not Animal's.



# SUBCLASS POLYMORPHISM – ACCESS METHOD NOT AVAILABLE IN PARENT

```
public class TestPolymorphism {
 public static void main(String[] args) {
 Animal a = new Bird();
 a.fly(); // Compile error: The method fly() is undefined for the type Animal
 }
}
```

□ So what to do if need to call subclass method that  
is not available in Parent class?

● Casting

```
public class TestPolymorphism {
 public static void main(String[] args) {
 Animal a = new Bird();
 ((Bird)a).fly(); // OK
 }
}
```



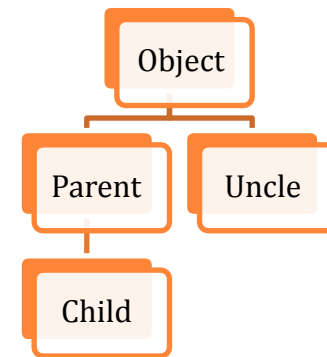
# CASTING AND CLASSES

- An instance of a child class can be assigned to a variable (field) of the parent class.
- variable references an subtype object can be cast down to its subclass type with an **explicit** cast.
- A runtime error occurs when explicitly casting an object to a type that it is not.
  - An object of type Parent cannot be cast to type Child.
- In the code shown in next page, the cast "(Uncle) object" is a runtime error
  - because at that time object holds an instance of the Child class, which is not of type (or subclass) of Uncle.



# CASTING AND CLASSES

```
class Casting {
 public static void main(String args[]) {
 Object object;
 Parent parent;
 Child child = new Child();
 Uncle uncle;
 parent = child;
 object = child;
 parent = (Parent) object; // explicit cast down
 child = (Child) object; // explicit cast down
 uncle = (Uncle) object; //Runtime exception
 }
}
```



## Output

java.lang.ClassCastException: Child: cannot cast to Uncle

- the cast "(Uncle) object" is a runtime error - because at that time object holds an instance of the Child class, which is not of type (or subclass) of Uncle.



# PRECAUTION WHILE DOWN CASTING

- We need to first check if the object is of that type.
- How?
  - The **java instanceof operator** is used to test whether the object is an instance of the specified type (class or subclass or interface).
  - `getClass()` method return the “class [ClassName]”
  - `getClass().getName()` return the class name.
- Let's revisit example of polymorphism.

```
public class TestPolymorphism {
 public static void main(String[] args) {
 Animal a = new Bird();
 if (a instanceof Bird) // can also use if (a.getClass().getName().equals("Bird"))
 ((Bird)a).fly(); // OK
 }
}
```



# Why we need Subclass Polymorphism

## **Scenario: Graphics Editor Application**

Imagine you're building a graphics editor application that allows users to draw, manipulate, and display different shapes. The application should support various shapes such as circles, rectangles, triangles, and squares, and it should allow new shapes to be added in the future without modifying existing code.

### **Requirements:**

- Draw Shapes: The application must be able to draw any shape.
- Add New Shapes: New shapes should be easily added without changing the core logic of the application.



# Solving the problem without Subclass Polymorphism

```
class shape { 6 usages 4 inheritors
 public void draw() { 1 usage 4 overrides
 System.out.println("Drawing a shape");
 }
}

class circle extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a circle");
 }
}

class rectangle extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a rectangle");
 }
}

class triangle extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a triangle");
 }
}

class square extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a square");
 }
}
```

```
public class Main {
 public static void main(String[] args) {

 // 2 circles, 3 rectangles, 4 triangles, 5 squares
 circle[] c = new circle[2];
 rectangle[] r = new rectangle[3];
 triangle[] t = new triangle[4];
 square[] sq = new square[5];

 for (int i = 0; i < 2; i++) {
 c[i] = new circle();
 c[i].draw();
 }
 for (int i = 0; i < 3; i++) {
 r[i] = new rectangle();
 r[i].draw();
 }
 for (int i = 0; i < 4; i++) {
 t[i] = new triangle();
 t[i].draw();
 }
 for (int i = 0; i < 5; i++) {
 sq[i] = new square();
 sq[i].draw();
 }
 }
}
```

# Solving the problem with Subclass Polymorphism

```
class shape { 6 usages 4 inheritors
 public void draw() { 1 usage 4 overrides
 System.out.println("Drawing a shape");
 }
}

class circle extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a circle");
 }
}

class rectangle extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a rectangle");
 }
}

class triangle extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a triangle");
 }
}

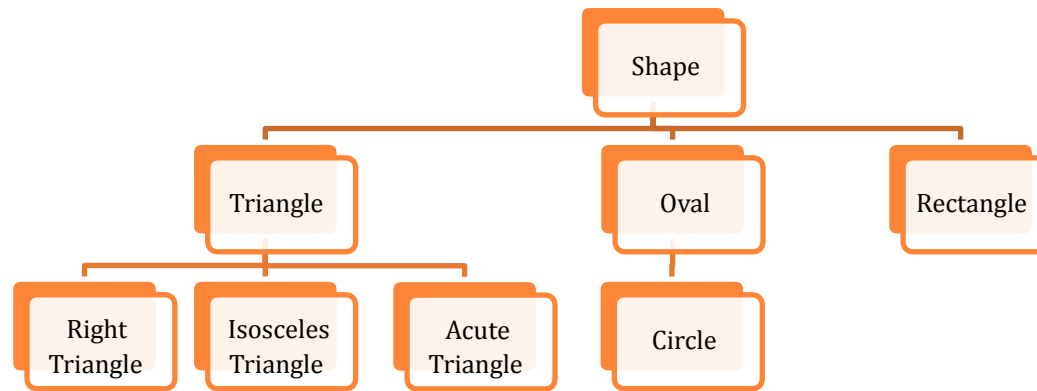
class square extends shape { 1 usage
 public void draw() { 1 usage
 System.out.println("Drawing a square");
 }
}
```

```
// 2 circles, 3 rectangles, 4 triangles, 5 squares
shape[] s1 = new shape[14];
for (int i = 0; i < 14; i++) {
 if (i < 2)
 s1[i] = new circle();
 else if (i < 5)
 s1[i] = new rectangle();
 else if (i < 9)
 s1[i] = new triangle();
 else
 s1[i] = new square();
 s1[i].draw();
}
```





# POLYMORPHISM – BENEFITS



- ❑ Assume the above hierarchy.
- ❑ Consider each class has a `draw()` method to draw the specific shape.
- ❑ Also consider a method that will take a Shape reference variable (which can hold a shape object or any of its subclass's object) as argument and call the draw method. E.g.

```
static void drawShape(Shape shape){
 shape.draw();
}
```



# POLYMORPHISM – BENEFITS

- What will happen if you call the method using a Triangle object or a Circle object.

```
drawShape(new Triangle()); // will call the draw method of Triangle class
drawShape(new Circle()); // will call the draw method of Circle class
```

- The same invocation can produce “many forms” of results
- We can override method in subclasses and which implementation to be used is decided at runtime depending upon the situation (i.e., data type of the real object)



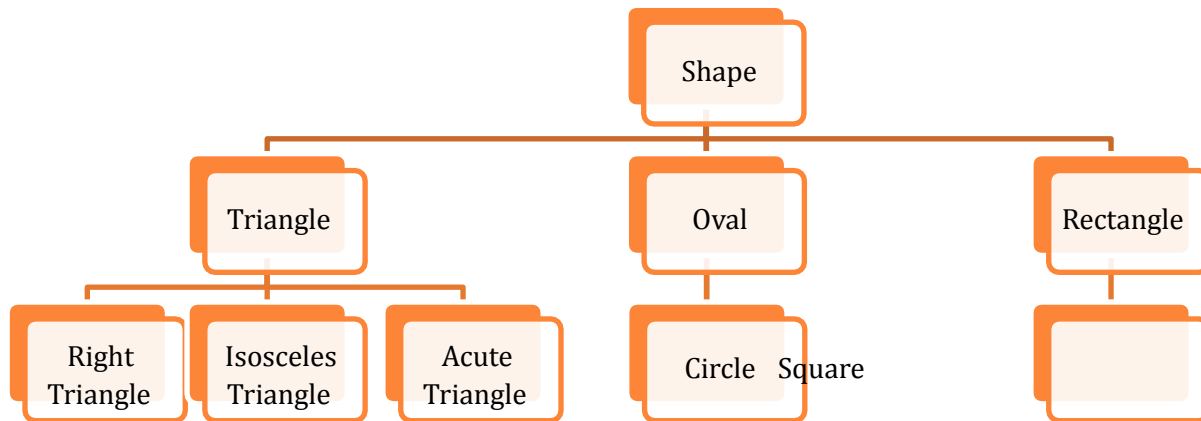
# POLYMORPHISM - BENEFITS

- Polymorphism enables programmers to deal with generalities and
  - let the execution-time environment handle the specifics.
- Programmers can command objects to behave in manners appropriate to those objects,
  - without knowing the types of the objects
  - (as long as the objects belong to the same inheritance hierarchy).



# POLYMORPHISM - BENEFITS

- Now think : - What needs to change in the old hierarchy or *drawShape(Shape shape)* method, if you add a new subtype “Square” as shown below.



- Polymorphism Promotes Extensibility
  - New object types that can respond to existing method calls can be
    - incorporated into a system **without requiring** modification of the base system.
    - Only client code **that instantiates new objects must be modified** to accommodate new types.



# STATIC & DYNAMIC BINDING

- Association of method definition to the method call is known as binding.
- 2 types:
  - static binding (also known as early binding).
  - dynamic binding (also known as late binding).



# STATIC BINDING

- The binding which can be resolved at compile time by compiler is known as static. Or
- When type of the object is determined at compiled time(by the compiler), it is known as static binding.
- All the static, private and final methods have always been bonded at **compile-time** .

## ● Why?

- Compiler knows that all such methods cannot be overridden and will always be accessed by object of local class.
- Hence compiler doesn't have any difficulty to determine object of class (local class for sure).
- That's the reason binding for such methods is static.



# DYNAMIC BINDING

- When compiler is not able to resolve the call/binding at compile time, such binding is known as Dynamic or late Binding.
- Example: Overriding
  - in overriding both parent and child classes have same method.
  - Thus while calling the overridden method, the compiler gets confused between parent and child class method.
  - The method is decided during runtime.



## Spring 2024 Question 2

### QUESTION 2

[3+2+1.5+1 MARKS]

Modify the following program by including/excluding the some codes without changing the highlighted parts.

```
public class Myparent {
 private int p;
 public final int myfunction() {
 return p*p;
 }
 public void set_p(int Q) {p = Q;}
 // Write your code here
}
public class Mychild extends Myparent {
 public Mychild(int K) { super(K); }

 public final int myfunction() {
 return p*p+1;
 }
 // write your code of myroot() that finds the square
 // root of p in class Myparent
 // write other necessary codes here
}
```

```
public class Mytest {
 public static void main(String[] args) {
 Myparent c1, c2;
 c1 = new Mychild(2);
 c2 = new Mychild();
 c2.set_p(2);
 int x = c2. myfunction ();
 double y = ((Mychild) c1).myroot(); // find square
 root of p in class Myparent
 System.out.println("x = " + x + ", y = " + y);
 }
}
```



## Spring 2024 Question 2

```
public class Myparent { 2 usages 1 inheritor
 private int p; 5 usages

 public Myparent(int K) { 2 usages
 p = K;
 }

 public final int myfunction() { 1 usage
 return p * p;
 }

 public void set_p(int Q) { 1 usage
 p = Q;
 }

 public double myroot() { 1 usage
 return Math.sqrt(p);
 }
}
```

```
public class Mychild extends Myparent { 3 usages

 public Mychild(int K){ 1 usage
 super(K);
 }

 public Mychild(){ 1 usage
 super(K: 0);
 }

 // public final int myfunction() {
 // return p*p+1;
 // }
}
```



# Spring 2024 Question 4

## QUESTION 4

[7.5 MARKS]

Suppose that you visit a village market where fresh vegetables and fishes are sold. The sellers sell their items with a profit of  $z\%$  of their production cost  $c$ . Typical items are given by the following Table:

| Food Items | Type $t$    | production cost $c$ per Kg | profit of $z\%$ |
|------------|-------------|----------------------------|-----------------|
| vegetable  | Spinach     | 20                         | 15              |
| vegetable  | Cauliflower | 25                         | 18              |
| fish       | Carp        | 300                        | 15              |
| fish       | medium      | 250                        | 20              |
| fish       | small       | 200                        | 25              |

The class `FoodItem` that includes type  $t$ , production cost  $c$  and profit  $z$  as public variables and a method `findprice()` is given as follows:

```
public class FoodItem {
 public double c, z;
 public String t;
 public double getprice(double amount){
 return c*amount*(1+z/100);
 }
}
```

## Spring 2024 Question 4

Next, two derived classes *Vegetable* and *Fish* are given as follows.

```
public class Vegetable extends FoodItem{
 public void setparameter(){
 if(t == "Spinach"){ c = 20; z = 15; }
 else if(t == "Cauliflower"){c = 25; z = 18;}
 }
 public Vegetable(String t){
 this.t = t;
 }
}
```

```
public class Fish extends FoodItem{
 public void setparameter(){
 if(t == "Carp"){ c = 20; z = 15; }
 else if(t == "medium"){c = 25; z = 20;}
 else if(t == "small"){c = 200; z = 25;}
 }
 public Fish(String t){
 this.t = t;
 }
}
```

Now as a programmer, test the above classes in the main method in a new class *MyTest* by finding your total purchase price if you buy 3Kg fish of type *small* and 2 Kg vegetable of type *Cauliflower*. [Make 2 objects of *FoodItems* and use the child class references to a *FoodItems* class object. Then call appropriate methods e.g., *setparameter*, *getprice*.]



## Spring 2024 Question 4

```
public class MyTest {
 public static void main(String[] args) {
 FoodItem fish = new Fish(t: "small");
 ((Fish)fish).setparameter();
 double fishPrice = fish.getprice(amount: 3);

 FoodItem vegetable = new Vegetable(t: "Cauliflower");
 ((Vegetable)vegetable).setparameter();
 double vegetablePrice = vegetable.getprice(amount: 2);

 double totalPrice = fishPrice + vegetablePrice;
 System.out.println("Total purchase price: " + totalPrice);
 }
}
```

## Fall 2023 Question 1(b)

(b) What is the output of following java codes:

```
public class Animal {
 public String color;
 public String name;
 public Animal() {
 System.out.println("Default animal");
 color = "Unknown";
 }
 public void showNameColor() {
 System.out.println("Color is: "+ color+" Name is: "+
name); }
 {
 System.out.println("Animal instance initialization ");
 }
}
```

```
public class Pokemon extends Animal
{
 public String name = "Pikachu";
 public String color = "Red";
}
public class AnimalTest {
 public static void main(String[] args) {
 Animal defaultAnimal = new
 Animal();
 Animal pk = new Pokemon();
 defaultAnimal.showNameColor();
 pk.showNameColor();
 }
}
```



## Fall 2023 Question 1(b)

(b) What is the output of following java codes:

```
public class Animal {
 public String color;
 public String name;
 public Animal() {
 System.out.println("Default animal");
 color = "Unknown";
 }
 public void showNameColor() {
 System.out.println("Color is: "+ color+" Name is: "+
name); }
 {
 System.out.println("Animal instance initialization ");
 }
}
```

```
public class Pokemon extends Animal
{
 public String name = "Pikachu";
 public String color = "Red";
}
public class AnimalTest {
 public static void main(String[] args) {
 Animal defaultAnimal = new
 Animal();
 Animal pk = new Pokemon();
 defaultAnimal.showNameColor();
 pk.showNameColor();
 }
}
```

```
Animal instance initialization
Default animal
Animal instance initialization
Default animal
Color is: Unknown Name is: null
Color is: Unknown Name is: null
```



## Fall 2023 Question 3

**Q3:** Consider the class named *Vehicle*.

[1+1+2  
+1+1]

```
class Vehicle {
 private String make;
 private String model;
 public Vehicle(String make, String model) {
 this.make = make;
 this.model = model;
 }
 public void start() {
 System.out.println("[Vehicle] The vehicle is starting.");
 }
}
```



## Fall 2023 Question 3

```
public void stop() {
 System.out.println("[Vehicle] The vehicle is stopping.");
}
public void drive() {
 System.out.println("[Vehicle] The vehicle is moving.");
}
}
```

Create a class named *Car* that inherits the *Vehicle* class.

The *Car* class must have the following attributes/methods:

- An additional attribute named *numberOfDoors* (data type: int, access modifier: private).
- A *constructor* that receives the values of *make* (String), *model* (String), *numberOfDoors* (int) as arguments and initializes the attributes.
- A *method* that overrides the method *drive()* of the parent class. This method invokes *drive()* of the parent class first and then prints “[Car] The car is moving.”.
- Another method named *honk()* (with access modifier: public and return type: void) that prints “[Car] Honk! Honk!”
- Now, consider the class named *Main* and write the output.

```
public class Main {
 public static void main(String[] args) {
 Vehicle car1 = new Car("make001", "model001", 4);
 car1.drive();
 car1.honk();
 }
}
```





## Fall 2023 Question 3

```
class Vehicle { 2 usages 1 inheritor
 private String make; 1 usage
 private String model; 1 usage

 public Vehicle(String make, String model) { 1 usage
 this.make = make;
 this.model = model;
 }

 public void start() { no usages
 System.out.println("[Vehicle] The vehicle is starting.");
 }

 public void stop() { no usages
 System.out.println("[Vehicle] The vehicle is stopping.");
 }

 public void drive() { 2 usages 1 override
 System.out.println("[Vehicle] The vehicle is moving.");
 }
}
```



## Fall 2023 Question 3

```
class Car extends Vehicle { 1 usage
 private int numberOfDoors; 1 usage

 public Car(String make, String model, int numberOfDoors) { 1 usage
 super(make, model);
 this.numberOfDoors = numberOfDoors;
 }

 @Override 2 usages
 public void drive() {
 super.drive();
 System.out.println("[Car] The car is moving.");
 }

 public void honk() { no usages
 System.out.println("[Car] Honk! Honk!");
 }
}
```



## Fall 2023 Question 3

```
public class Main {
 public static void main(String[] args) {
 Vehicle car1 = new Car(make: "make001", model: "model001", numberOfDoors: 4);
 car1.drive();
 // car1.honk(); // This line will cause a compilation error
 }
}
```

[Vehicle] The vehicle is moving.

[Car] The car is moving.



## Fall 2023 Question 4

**Q4:** Think about your own startup **Uthao** which is a ride sharing program using Java. Now due to our country's rules and regulations, vehicles on your platform must abide by the *speedLimit* at 80 km/h. [2+1.5+1.5+1]

(a) Create a class name **Ride** that contains an attribute *speedLimit* which

- Cannot get changed by its child classes
- Cannot get changed once assigned
- Will contain an Integer value
- Must be **static**

Now create the following child classes of **Ride**:

- Bike
- Car
- Microbus

Each child of Ride (that is, Bike, Car and Microbus) will receive a penalty for exceeding the *speedLimit*.

All the child classes have the following

*initial\_speed* and *acceleration*:

- Bike: *initial\_speed* 20 km/h , *acceleration* 2 km/h
- Car: *initial\_speed* 40 km/h, *acceleration* 10 km/h
- Microbus: *initial\_speed* 15 km/h, *acceleration* 5 km/h

Now do the following tasks:

(b) Create a method named *getHighestAccelerationTime()* ( return type: double ) which will find out the time needed for a ride to reach the *speedLimit* by using the formula :  $v = u + at$

## Fall 2023 Question 4

(c) Create a method named *calculateFine(int hour)* (return type: double) which will calculate the fine for each vehicle by following the implementation below:

- **Bike:** Base fine will be 50 TK and for each hour exceeding **speedLimit**, it will add an extra 100 TK
- **Car:** Base fine will be 100 TK and for each hour exceeding **speedLimit**, it will add an extra 150 TK
- **Microbus:** Total fine will be 3000 TK for just exceeding **speedLimit**.

(d) Find out the output of the following Code:

```
public class Uthao {
 public static void main(String[] args) {
 Ride car = new Car();
 System.out.println(car.calculateFine(hour: 10));
 }
}
```



## Fall 2023 Question 4

```
class Ride { 4 usages 3 inheritors
 public static final int speedLimit = 80; 1 usage

 public int initial_speed; 2 usages
 public int acceleration; 2 usages

 public Ride(int initial_speed, int acceleration) { 3 usages
 this.initial_speed = initial_speed;
 this.acceleration = acceleration;
 }

 public double getHighestAccelerationTime() { no usages
 return (double) (speedLimit - initial_speed) / acceleration;
 }

 public double calculateFine(int hour) { 1 usage 3 overrides
 return 0;
 }
}
```





## Fall 2023 Question 4

```
class Bike extends Ride { no usages
 public Bike() { no usages
 super(initial_speed: 20, acceleration: 2);
 }

 @Override 1 usage
 public double calculateFine(int hour) {
 return 50 + 100 * hour;
 }
}

class Car extends Ride { 1 usage
> public Car() { super(initial_speed: 40, acceleration: 10); }

 @Override 1 usage
> public double calculateFine(int hour) { return 100 + 150 * hour; }
}

class Microbus extends Ride { no usages
> public Microbus() { super(initial_speed: 15, acceleration: 5); }

 @Override 1 usage
> public double calculateFine(int hour) { return 3000; }
}
```



## Fall 2023 Question 4

```
public class Uthao {
 public static void main(String[] args) {
 Ride car = new Car();
 System.out.println(car.calculateFine(hour: 10));
 }
}
```

1600.0





# REFERENCE

- Java:Complete Reference Chapter 7,8
- Online Reference:
  - <http://www.javatpoint.com/super-keyword>
  - <https://www.tutorialspoint.com/java/>
  - <https://docs.oracle.com/javase/tutorial/>

